

AL & Big Data Lab Report



Prepared by:

Vincent Mwenda Mworio

Instructor:

Prof. Frédéric Ros

Date:

1st December 2025

Academic Year 2025/2026

Table of Contents

Lab 1 - K-Means Clustering	3
Objective	3
Theory	3
Code	3
Implementation on Synthetic Data	3
Determining the Optimal Number of Clusters	4
Applying K-Means to the Iris Dataset	5
Exploring Limitations with Non-Spherical Data	6
Cluster Evaluation (Silhouette Score)	8
Questions for Discussion	8
Conclusion	9
Lab 2 - DBSCAN Clustering	10
Objective	10
Theory	10
Code	11
Implementation on Synthetic Non-Spherical Data	11
Parameter Sensitivity	12
Comparing to K-Means	14
DBSCAN on the Iris Dataset (Real-World Multidimensional Data)	15
Data with Varying Density	16
Using Silhouette Score and Adjusted Rand Index to Evaluate Clustering Quality	17
HDBSCAN on Varying-Density Data	19
Questions for Discussion	20
Conclusion	20
Lab 3 - Hierarchical Clustering	21
References	22

Lab 1 - K-Means Clustering

Objective

This experiment aims to explore how the K-Means clustering algorithm groups data into clusters based on similarity.

The lab focuses on:

- Understanding the K-Means process.
- Applying it to a simple 2D dataset.
- Finding the best number of clusters using the Elbow method.
- Testing it on a real-world dataset (Iris).
- Observing its strengths and weaknesses.

Theory

K-Means is an unsupervised learning algorithm used to discover natural groupings (clusters) in data. It partitions the data into K clusters such that points within a cluster are more similar to each other than to those in other clusters.

The algorithm follows these main steps:

1. **Start:** Choose K random points as initial centroids.
2. **Assign:** Allocate each data point to the nearest centroid.
3. **Update:** Recalculate centroids as the average of all points in each cluster.
4. **Repeat:** Continue until cluster centers no longer move significantly.

The algorithm minimizes the **sum of squared distances** between each point and its assigned centroid (called *inertia*).

Code

The K-Means clustering code can be found at the following GitHub link: [K-Means_code](#)

Implementation on Synthetic Data

To visualize how K-Means behaves, a simple 2D dataset with three clusters was created using `make_blobs()`.

The model successfully identified three distinct clusters in the dataset. The centroids, represented by black “X” symbols, indicate the center of each cluster. Each color in the plot corresponds to a separate cluster, clearly showing separation between the data groups and confirming that the K-Means algorithm effectively grouped similar data points together.

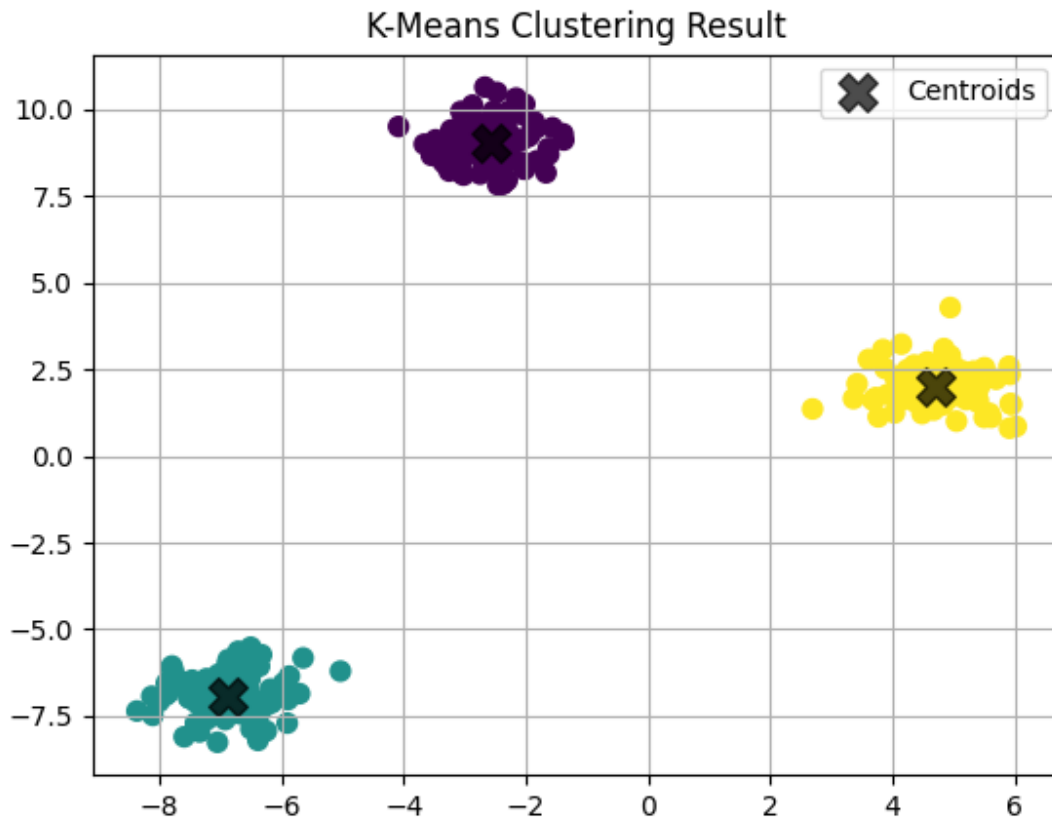


Figure 1. *K-Means clustering result on a 2D synthetic dataset.*

Determining the Optimal Number of Clusters

The Elbow Method was used to determine the most appropriate value for K. It measures how the total inertia (within-cluster sum of squares) changes as K increases.

As K grows, inertia naturally decreases because clusters become smaller and data points move closer to their respective centroids. However, beyond a certain point, the improvement becomes minimal hence additional clusters start capturing random noise rather than meaningful structure. Moreover, using too many clusters can lead to overfitting, where the model fits the data too precisely and loses generalization ability.

Therefore, the optimal value of K is chosen at the “elbow point” where adding more clusters no longer results in a significant decrease in inertia. This choice balances accuracy with simplicity, preventing the model from overfitting.

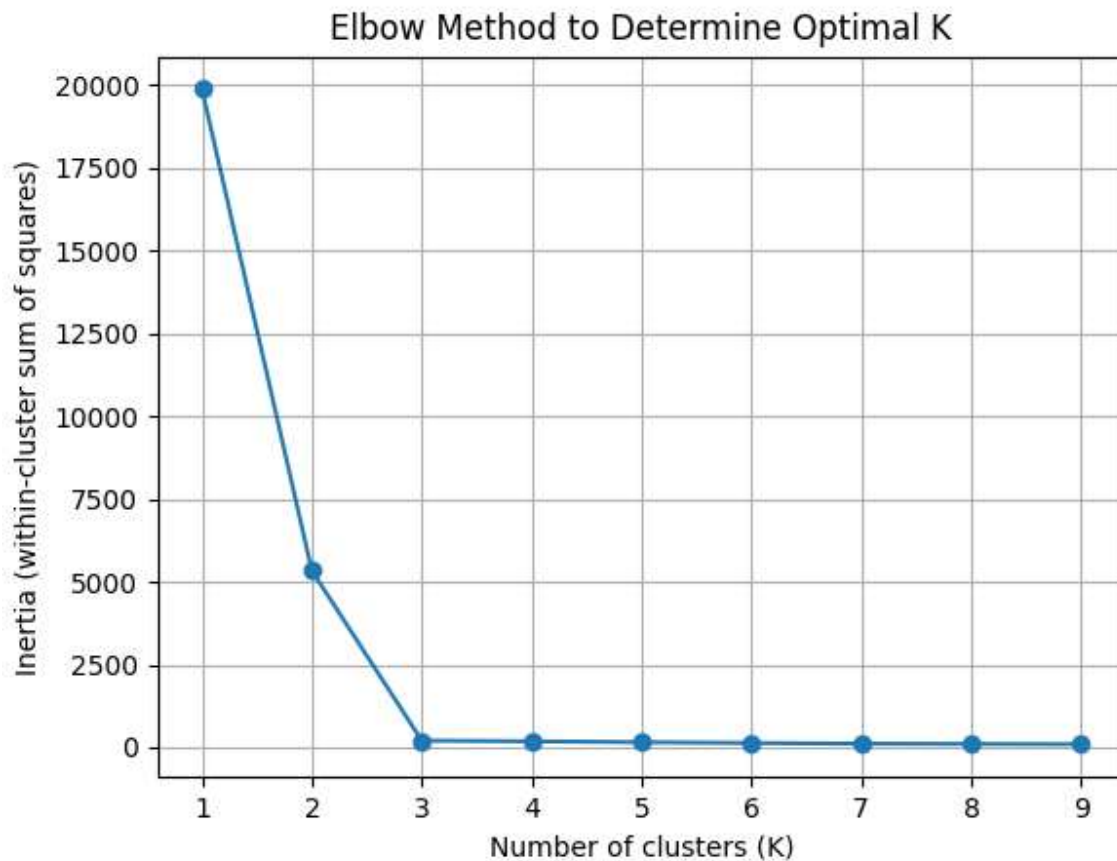


Figure 2. Elbow Method used to determine the optimal number of clusters.

Applying K-Means to the Iris Dataset

The algorithm was tested on the Iris flower dataset, which contains 150 samples and four numerical features.

Before clustering, the data was standardized using `StandardScaler()` to ensure that all features contributed equally to the distance calculations. This transformation scales each feature to have a mean of 0 and a standard deviation of 1, making the variables comparable across dimensions.

Principal Component Analysis (PCA) was also applied to reduce the data from four dimensions to two. This dimensionality reduction technique preserves most of the variance in the dataset while enabling effective visualization of the clustering results in a 2D plot.

K-Means successfully identified three major groups, roughly corresponding to the three Iris species. However, some overlap occurred between the purple cluster (versicolor) and the yellow cluster (virginica), indicating that the K-Means algorithm can struggle when clusters overlap or are not linearly separable in feature space.

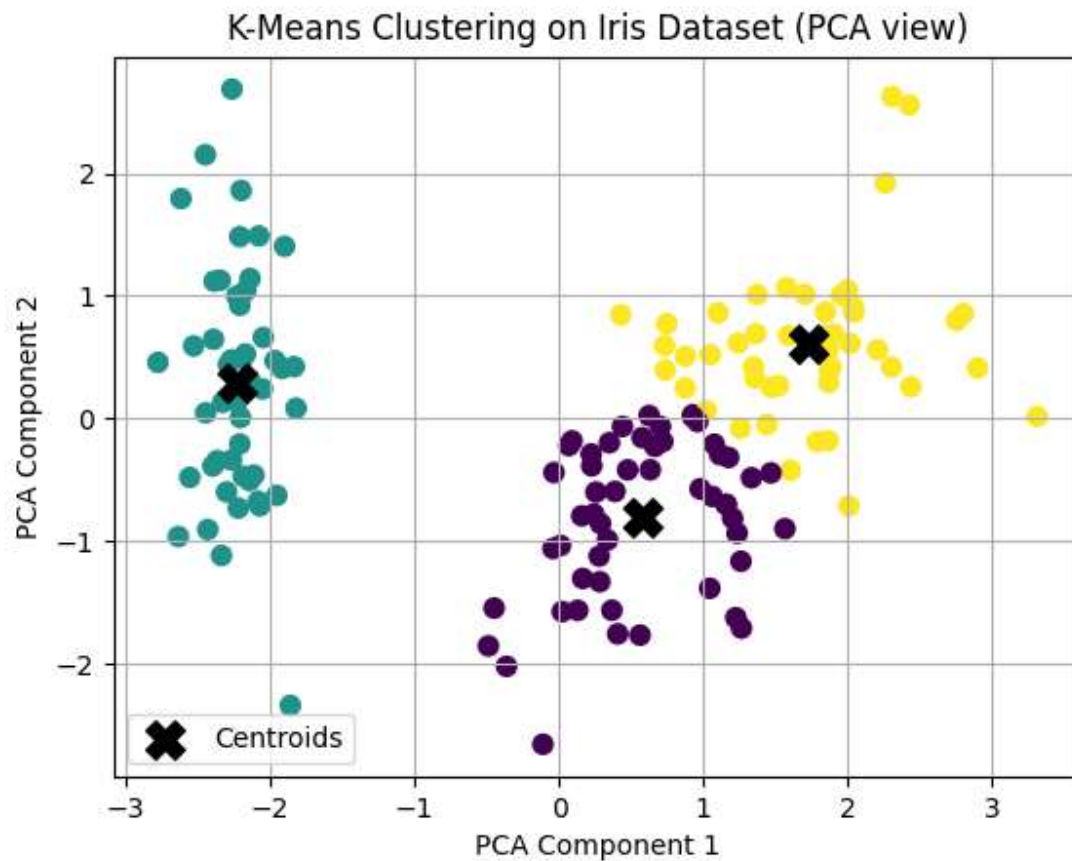


Figure 3. *K-Means clustering on the Iris dataset (2D PCA projection).*

Exploring Limitations with Non-Spherical Data

To examine how K-Means performs on irregularly shaped datasets, two synthetic datasets were generated. One was shaped like interlocking moons and another like concentric circles.

In both cases, the algorithm failed to correctly separate the curved clusters because K-Means relies on Euclidean distance to assign points to the nearest centroid. As a result, it naturally forms circular cluster boundaries. This assumption works well for spherical data but performs poorly when clusters are non-linear or non-spherical. Consequently, K-Means misclassifies several data points in regions where the clusters bend or overlap.

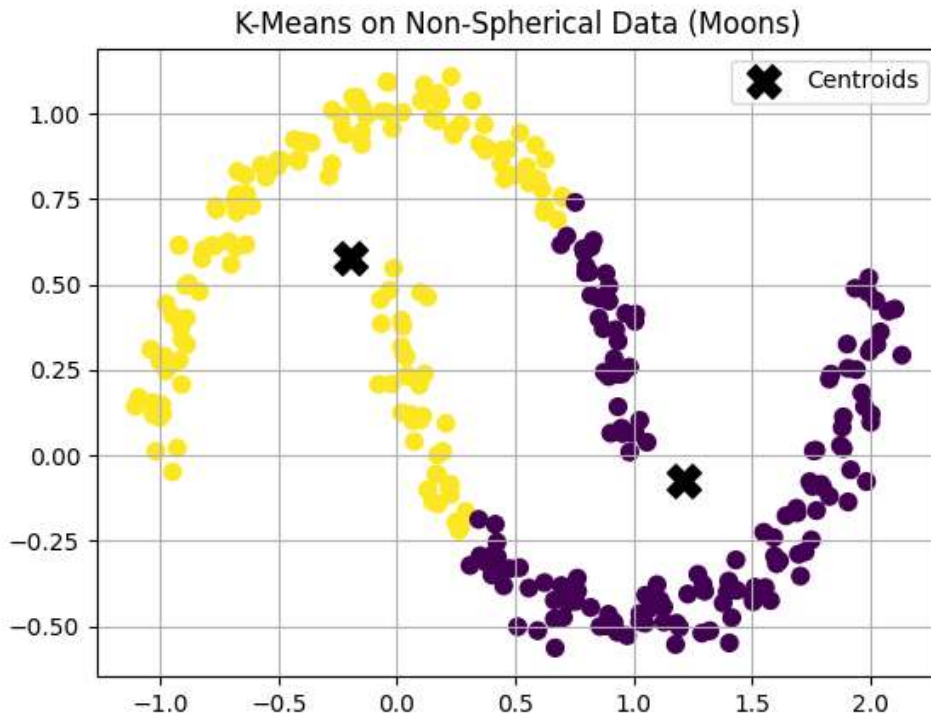


Figure 4. *K-Means clustering on non-spherical data (two interlocking moons)*

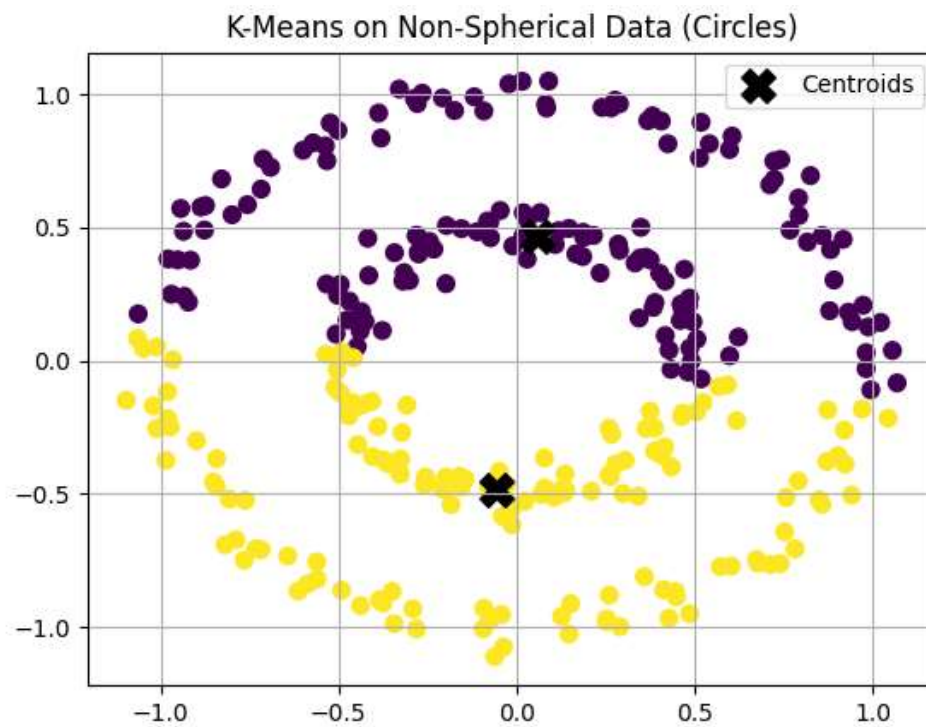


Figure 5. *K-Means clustering on non-spherical data (concentric circles)*

Cluster Evaluation (Silhouette Score)

The Silhouette Score measures how well each data point fits within its assigned cluster compared to other clusters. The score ranges from -1 to 1, where a value closer to 1 indicates that the data points are well clustered, while lower or negative values suggest poor separation or incorrect clustering. The highest silhouette value was observed at $K = 2$, indicating that the data is most cleanly divided into two distinct clusters. However, $K = 3$ still provides a reasonable structure that aligns with the known Iris species groupings.



Figure 6. *Silhouette Score versus number of clusters for the Iris dataset.*

Questions for Discussion

1. What are the limitations of K-Means?
 - K-Means requires specifying K in advance, assumes spherical and equally sized clusters, and performs poorly on irregular, overlapping, or outlier-heavy data.
2. What happens if clusters are not spherical?
 - K-Means forces circular boundaries, so it struggles with curved or uneven clusters, often splitting points from the same true group or grouping distant points together incorrectly.
3. Does K-Means always find the best result?
 - No. K-Means can get stuck in a local minimum, meaning it may find a suboptimal solution depending on where the centroids were initially placed.
4. What if centroids are initialized poorly?

- Poor initialization can lead to unbalanced or incorrect clusters and may also cause slower convergence. The K-Means++ method reduces this issue by choosing well-spaced initial centroids for more stable and accurate results.

Conclusion

K-Means is one of the most widely used and intuitive algorithms for identifying structure in data. It performs well on compact, well-separated clusters but struggles with irregular, overlapping, or non-spherical shapes. Overall, it serves as an excellent starting point for clustering analysis, though more advanced algorithms such as DBSCAN or Gaussian Mixture Models (GMMs) are better suited for complex or non-linear datasets.

K-Means is most commonly applied in customer segmentation, image compression, document grouping, and market analysis, where cluster boundaries are approximately convex and the number of groups can be estimated beforehand.

Lab 2 - DBSCAN Clustering

Objective

This experiment aims to explore how the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm discovers clusters by local density and identifies outliers. The lab focuses on:

- Understanding DBSCAN's core ideas and parameters (**eps**, **min_samples**) and the roles of core, border, and noise points.
- Applying DBSCAN to a non-spherical 2D dataset (two moons) and examining parameter sensitivity.
- Comparing DBSCAN with K-Means on the same data to highlight shape and outlier handling differences.
- Running DBSCAN on the Iris dataset (with PCA for visualization) and counting clusters vs. noise points.
- Noting practical limitations (single global **eps**, varying densities) and when alternatives like HDBSCAN may be preferable.

Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups together points closely packed in space, while marking isolated points as noise. It defines clusters based on two parameters:

- ϵ (eps): the neighborhood radius around a point.
- min_samples: the minimum number of points required to form a dense region.

Conceptually, the method considers a neighborhood of radius ϵ around each point in the dataset. If this neighborhood contains at least min_samples points, the point is classified as a **core point**. Points that lie within the ϵ -neighborhood of a core point but do not themselves meet the density requirement are referred to as **border points**. Points that are neither core nor border points are designated as **noise** or **outliers**.

Clusters are then constructed by connecting core points whose neighborhoods overlap, thereby forming continuous regions of high density. Unlike K-Means, DBSCAN does not require specifying the number of clusters and can identify clusters of arbitrary shapes, making it robust for non-spherical data and outlier detection. However, its performance depends strongly on suitable ϵ and min_samples values.

Code

The DBSCAN clustering code can be found at the following GitHub link: [DBSCAN_code](#)

Implementation on Synthetic Non-Spherical Data

A two-dimensional non-spherical dataset was generated using the `make_moons()` function with 300 samples and a noise level of 0.05. This dataset was selected because it cannot be effectively separated by algorithms that assume spherical clusters, such as K-Means. The DBSCAN algorithm was applied with parameters $\epsilon = 0.2$ and `min_samples = 5` to identify the two crescent-shaped clusters.

These parameter values were selected empirically through visual inspection of the distance distribution, as they provided a balance between accurately detecting the two main clusters and minimizing the number of points classified as noise.

A smaller ϵ value tends to fragment the data into numerous small clusters, often increasing the number of noise points and resulting in underfitting, where the model is too rigid to capture the full cluster structure. Conversely, an excessively large ϵ may merge distinct regions into a single cluster, leading to overfitting, where the model becomes too general and fails to preserve meaningful boundaries. Similarly, a low `min_samples` value allows small or spurious clusters to form, while a high `min_samples` value enforces stricter density requirements that may exclude legitimate but less dense clusters.

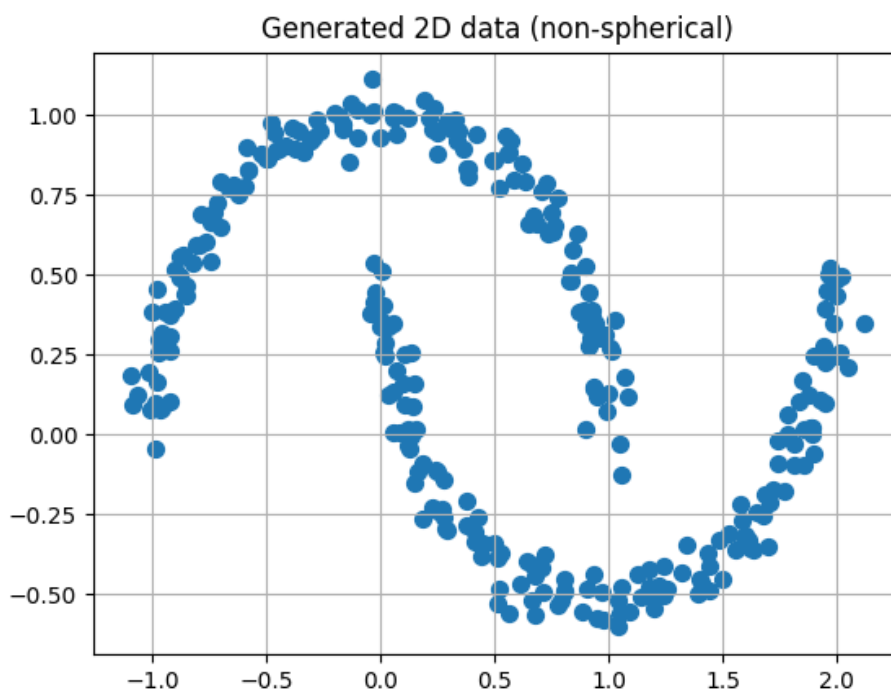


Figure 7. *Generated two-dimensional non-spherical dataset (two-moons) created using the `make_moons()` function.*

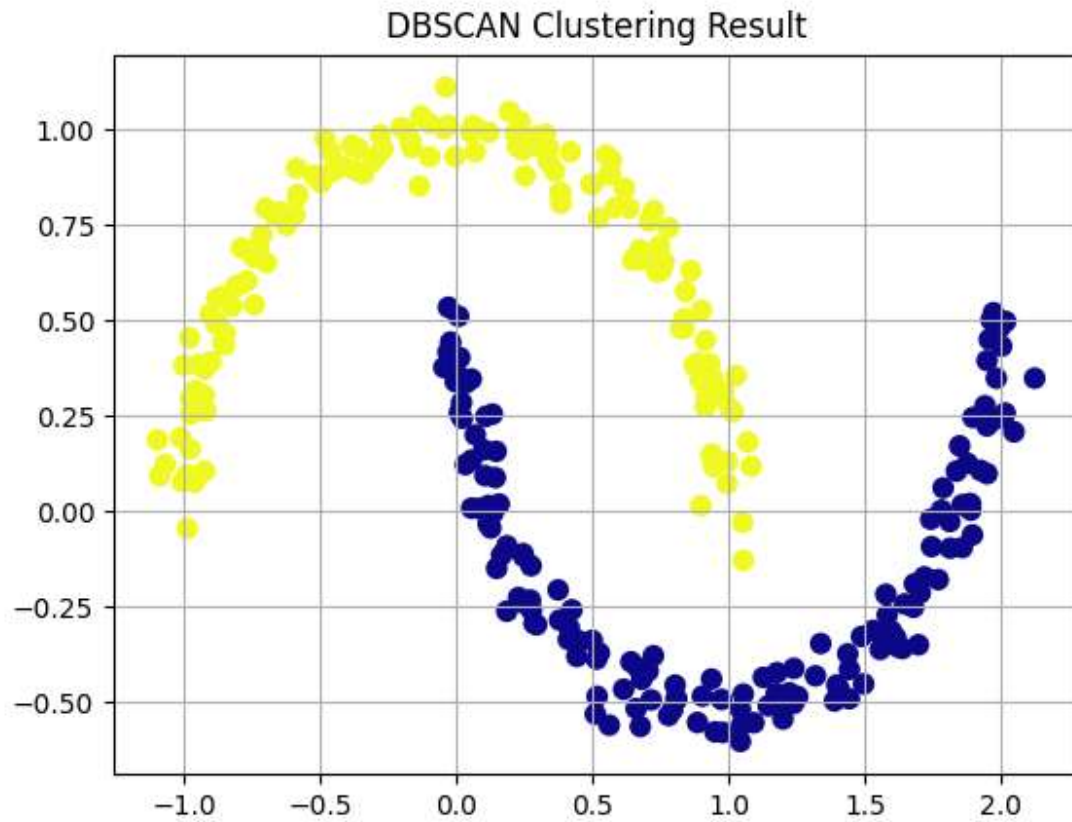


Figure 8. DBSCAN clustering result on the two-moons dataset using parameters $\epsilon = 0.2$ and $\text{min_samples} = 5$.

Parameter Sensitivity

To examine the influence of DBSCAN's parameters, experiments were conducted by varying ϵ and min_samples independently.

As ϵ increased, the neighborhood radius expanded, causing the algorithm to transition from detecting multiple small, fragmented clusters (when ϵ was small) to merging the two crescent-shaped structures into a single cluster (when ϵ was large). Smaller ϵ values led to excessive fragmentation and noise (underfitting), while larger ϵ values caused overgeneralization and cluster merging (overfitting).

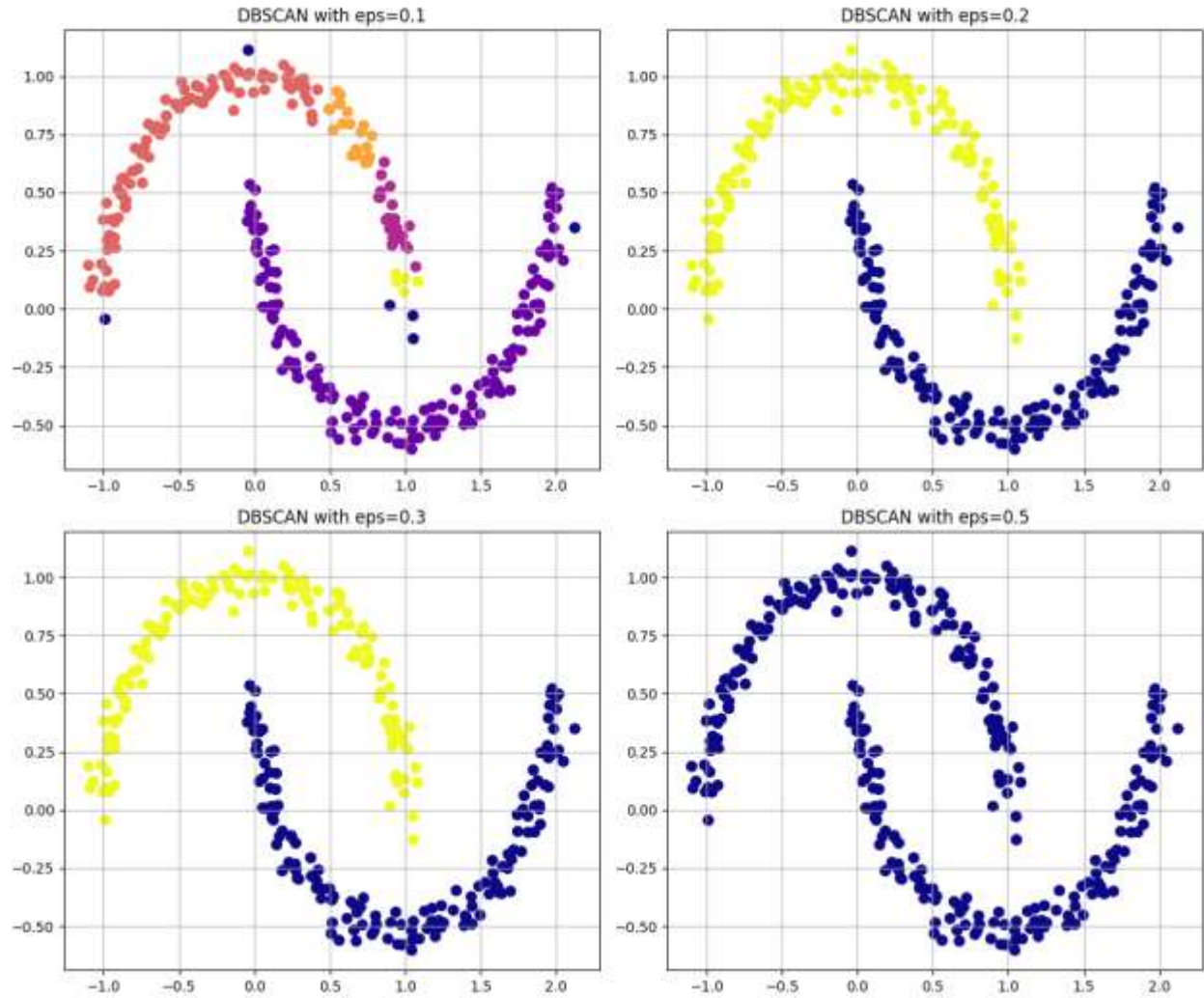


Figure 9. DBSCAN results with varying ϵ values (0.1, 0.2, 0.3, 0.5) and fixed $\text{min_samples} = 5$.

Similarly, increasing min_samples made the density criterion progressively stricter, reducing the number of core points and increasing noise, especially at higher values. Smaller min_samples values (e.g., 3–5) produced stable and coherent clusters, while very large values (e.g., 20) led to overly conservative clustering that excluded many valid points at the edges of each crescent.

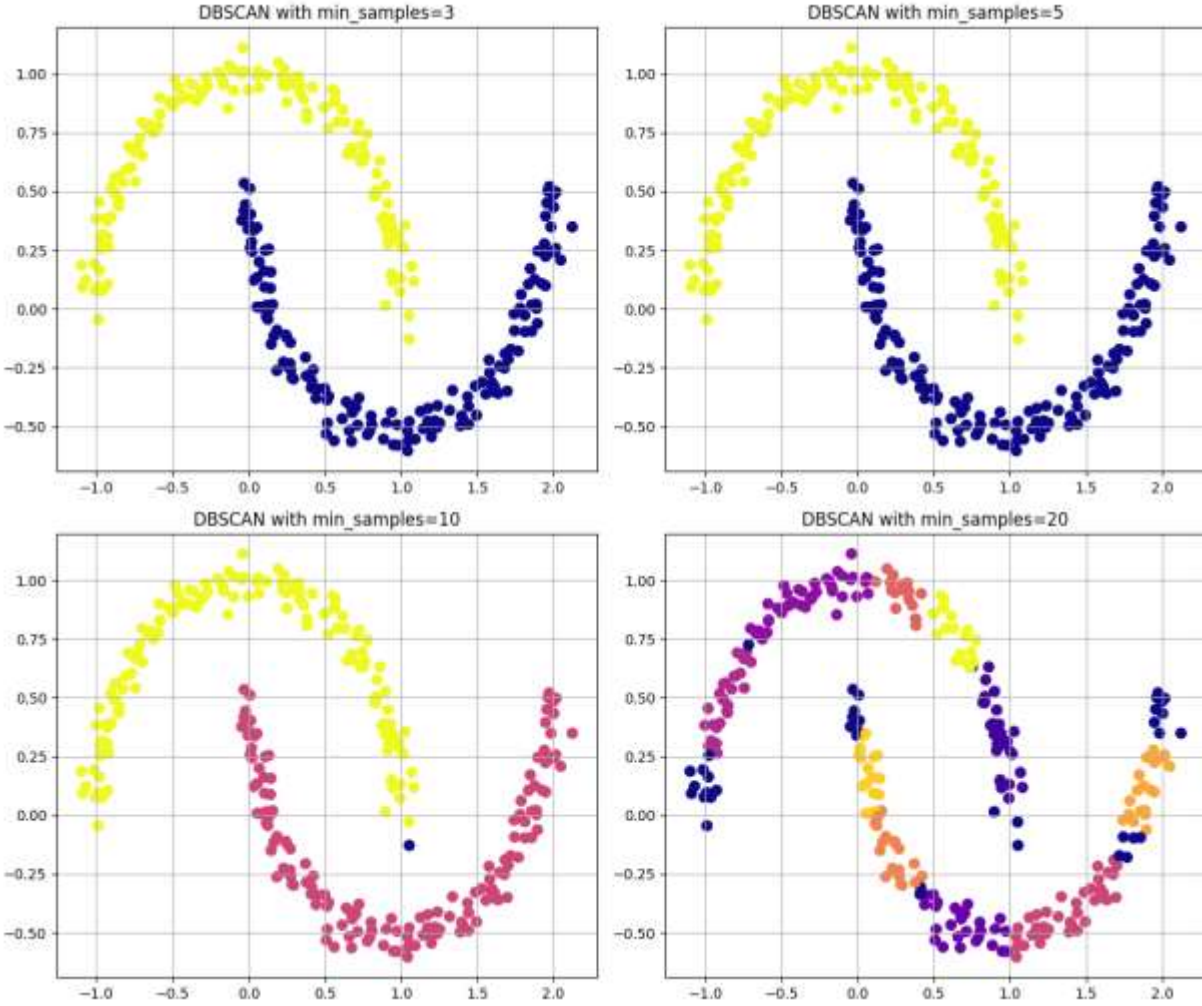


Figure 10. DBSCAN results with varying `min_samples` values (3, 5, 10, 20) and fixed $\epsilon = 0.2$.

An intermediate configuration ($\epsilon = 0.2$, `min_samples` = 5) yielded the most coherent and balanced clustering.

Comparing to K-Means

The same non-spherical two-moons dataset was also clustered using K-Means with $k = 2$. K-Means divided the data into two regions using straight, centroid-based boundaries that do not follow the curved structure of the dataset. As a result, several points near the junction of the two crescents were misclassified due to the algorithm's assumption of spherical cluster geometry and its reliance on Euclidean distance to fixed centroids.

DBSCAN, in contrast, accurately identified the two curved clusters without requiring the number of clusters in advance. Its density-based approach captured the true manifold of the data and correctly separated the clusters while labeling a few boundary points as noise. This result demonstrates DBSCAN's ability to model irregular and non-convex shapes that K-Means cannot effectively represent.

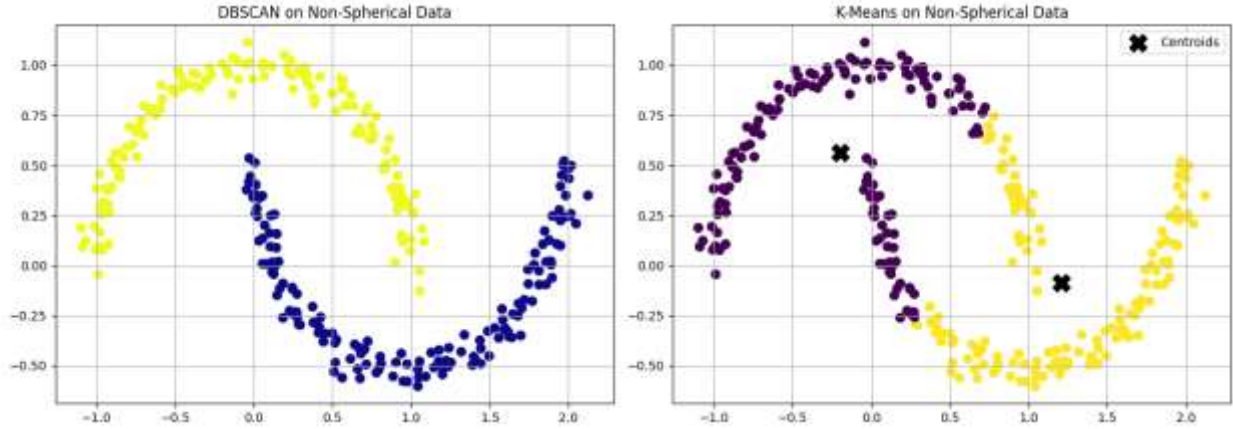


Figure 11. Comparison of clustering performance on the non-spherical two-moons dataset between DBSCAN ($\epsilon = 0.2$, $\text{min_samples} = 5$) on the left and K-Means ($k = 2$) on the right.

DBSCAN on the Iris Dataset (Real-World Multidimensional Data)

The Iris dataset was used to evaluate DBSCAN on real-world, multi-dimensional data. The feature variables were standardized using StandardScaler to ensure equal contribution of all attributes to the distance metric. Dimensionality reduction was then performed using Principal Component Analysis (PCA) to project the four-dimensional data into two principal components for visualization purposes only.

DBSCAN was applied to the standardized data using parameters $\epsilon = 0.6$ and $\text{min_samples} = 5$. The algorithm identified several compact regions of high density corresponding to potential clusters while labeling certain ambiguous samples as noise. The resulting two-dimensional PCA projection revealed visually separable groups that broadly corresponded to the Iris species, although some overlap occurred between Iris versicolor and Iris virginica, which are known to have similar feature distributions.

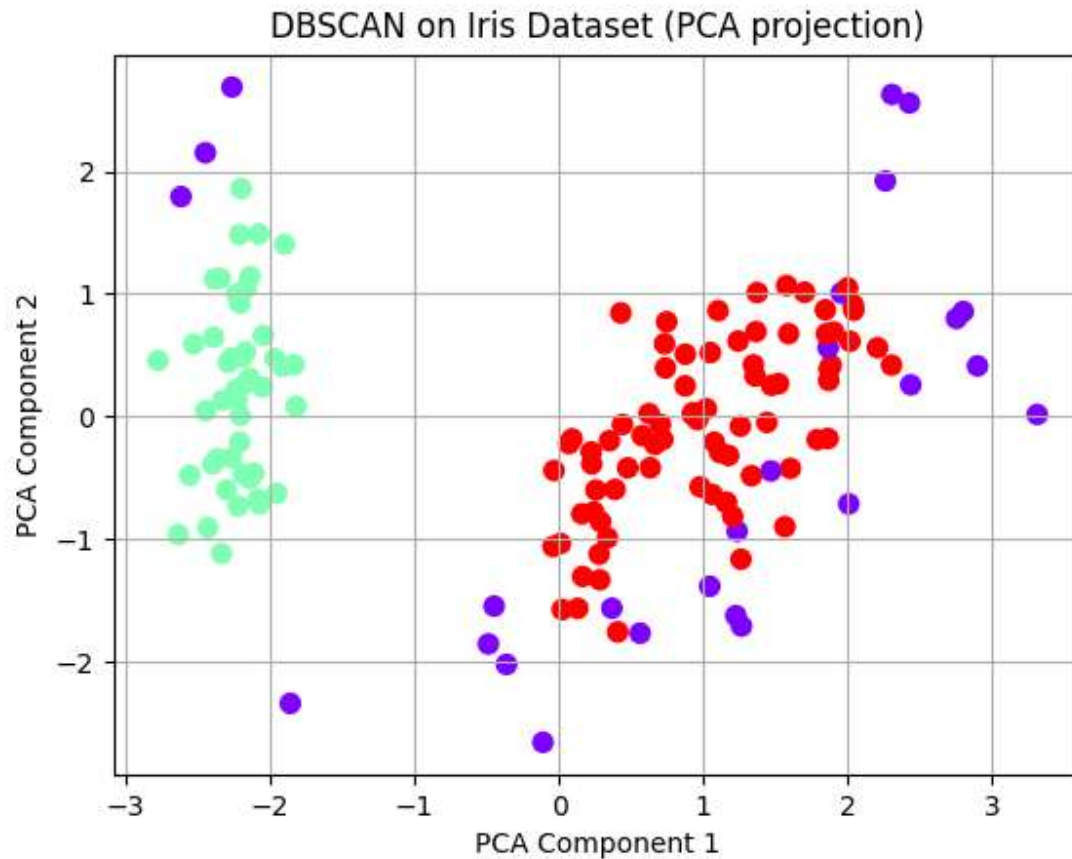


Figure 12: DBSCAN clustering of the Iris dataset visualized after PCA projection ($\epsilon = 0.6$, $\text{min_samples} = 5$).

Data with Varying Density

To examine DBSCAN's behaviour when cluster densities differ, a synthetic dataset was generated using the `make_blobs()` function with cluster standard deviations of $[0.5, 1.5, 0.3]$, creating three groups of unequal density. The algorithm was executed with parameters $\epsilon = 0.8$ and $\text{min_samples} = 5$.

The resulting plot shows that DBSCAN correctly identifies the two denser clusters but partially merges or misclassifies points in the sparser, more diffuse cluster. This occurs because DBSCAN applies a single global density threshold (ϵ), which cannot simultaneously accommodate both dense and sparse regions. Consequently, points belonging to low-density clusters are either labeled as noise or absorbed into neighboring clusters.

This experiment highlights DBSCAN's sensitivity to varying density and motivates the use of adaptive approaches such as HDBSCAN, which can handle non-uniform density distributions by adjusting neighborhood thresholds locally.

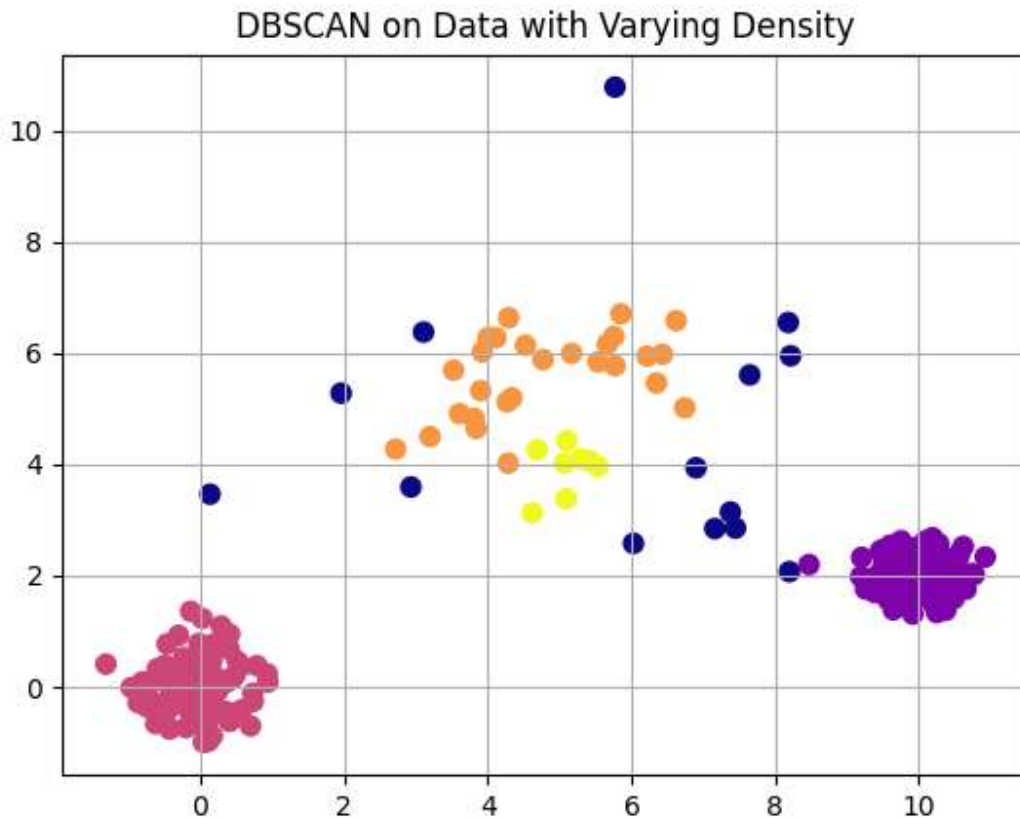


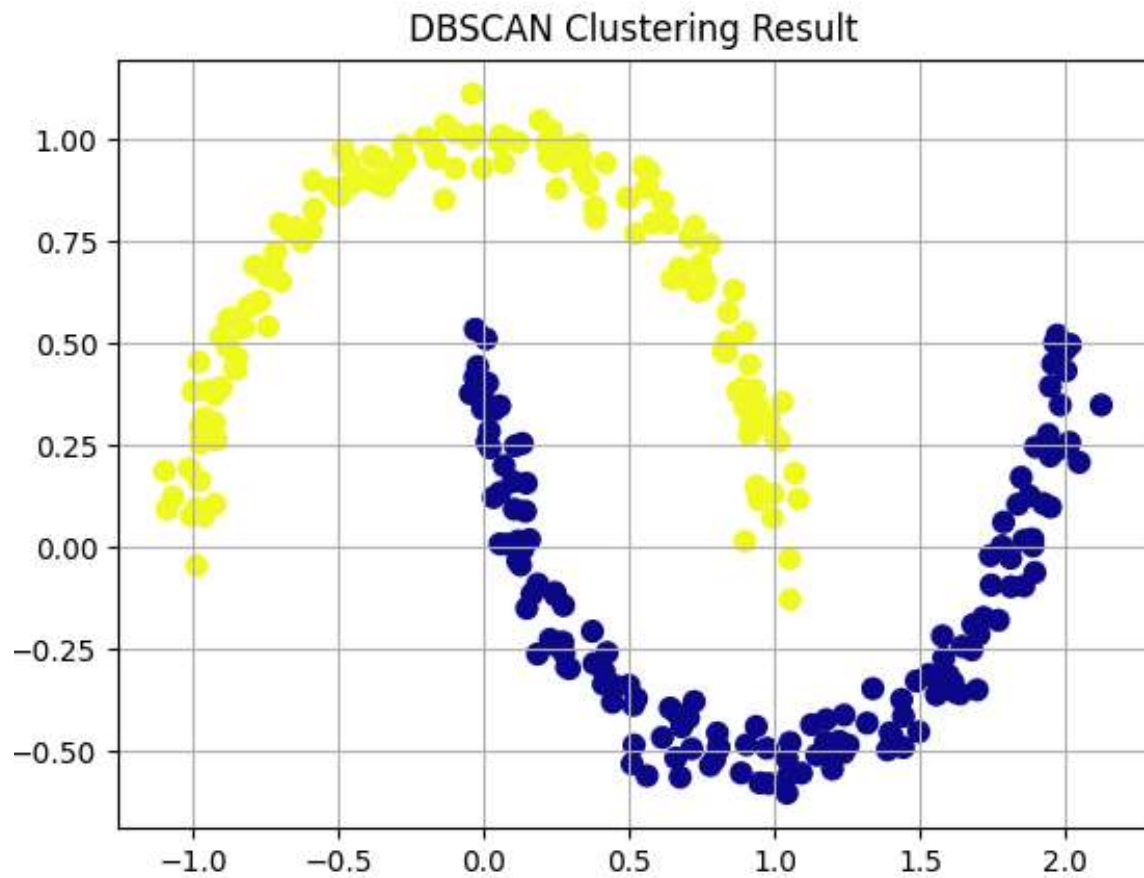
Figure 13: DBSCAN clustering of the Iris dataset visualized after PCA projection ($\epsilon = 0.6$, $\text{min_samples} = 5$).

Using Silhouette Score and Adjusted Rand Index to Evaluate Clustering Quality

To quantitatively assess DBSCAN's performance, two standard clustering evaluation metrics were computed on the two-moons dataset: the Silhouette Score and the Adjusted Rand Index (ARI).

The Silhouette Score of 0.328 indicates moderate cluster separation. Points within each cluster are generally closer to one another than to points in other clusters. However, some points near the edges of the crescent-shaped regions lie close in Euclidean distance to points in the neighboring cluster. Because the Silhouette metric evaluates separation based purely on straight-line distances, it penalizes these boundary points even though they are correctly assigned. This results in a moderate score, which is expected for non-spherical data such as the two-moons structure.

The Adjusted Rand Index (ARI) achieved a perfect score of 1.000, confirming that DBSCAN's cluster assignments exactly matched the true underlying structure of the dataset. This result demonstrates DBSCAN's strength in identifying complex, non-spherical cluster shapes that are difficult for centroid-based algorithms such as K-Means to represent accurately.



```
Run 2. dbscan x
Number of clusters <Moons Dataset>: 2
Number of noise points <Moons Dataset>: 0
Silhouette Score: 0.328
Adjusted Rand Index: 1.000
Process finished with exit code 0
```

Figure 14: Quantitative evaluation of DBSCAN performance on the two-moons dataset ($\epsilon = 0.2$, $\text{min_samples} = 5$).

HDBSCAN on Varying-Density Data

To address DBSCAN's limitation in handling clusters of unequal density, the same synthetic dataset with varying cluster standard deviations $[0.5, 1.5, 0.3]$ was analyzed using HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise). Unlike DBSCAN, HDBSCAN does not rely on a single global ϵ value; instead, it builds a hierarchy of density levels and extracts stable clusters based on their persistence across scales.

With parameters `min_cluster_size = 15` and `min_samples = 5`, HDBSCAN successfully detected three main clusters and a small number of noise points. Compared to DBSCAN, which struggled to separate the sparser middle group, HDBSCAN adaptively adjusted to local density variations, resulting in more accurate cluster boundaries and reduced misclassification of low-density points as noise.

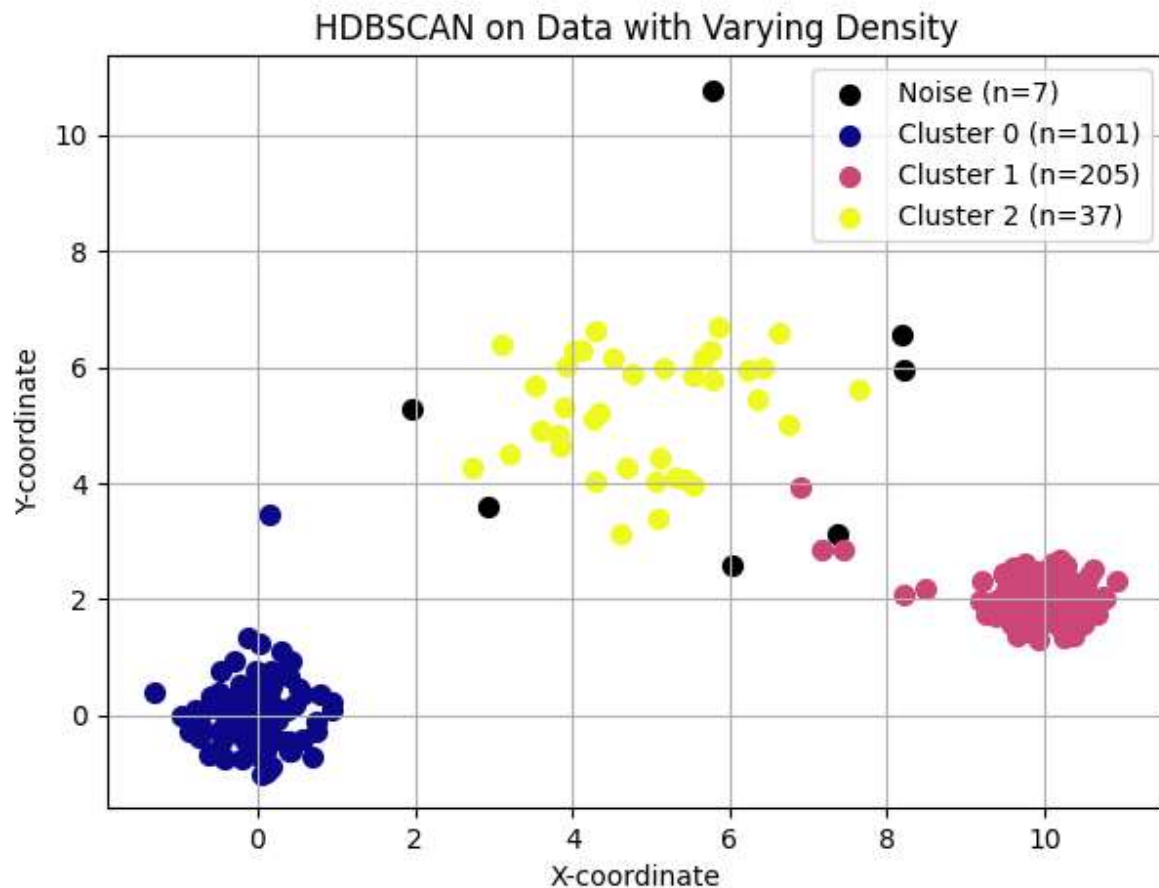


Figure 15: HDBSCAN clustering on data with varying density (`min_cluster_size = 15`, `min_samples = 5`).

Questions for Discussion

1. What are the advantages of DBSCAN over K-Means?
 - DBSCAN does not require the number of clusters to be specified in advance and can discover clusters of arbitrary shape, such as non-spherical or irregular structures that K-Means cannot represent. It is also able to detect outliers and label them as noise.
2. What are the limitations of DBSCAN?
 - DBSCAN is sensitive to its parameters ϵ (epsilon) and `min_samples`. Small changes in these values can significantly affect the resulting clusters. It also struggles with datasets containing clusters of varying density, since a single global ϵ cannot accommodate both dense and sparse regions simultaneously. In addition, DBSCAN's performance can degrade on very large or high-dimensional datasets because distance measures become less meaningful in such spaces.
3. Can DBSCAN identify noise in the Iris dataset?
 - Yes. DBSCAN labels points that do not belong to any sufficiently dense region as noise (label = -1).
4. Would DBSCAN work well in high-dimensional datasets?
 - Not very well. In high-dimensional spaces, the notion of “distance” becomes less informative due to the curse of dimensionality, making it difficult to define meaningful density thresholds. As a result, DBSCAN may either classify most points as noise or merge clusters incorrectly. Dimensionality reduction techniques such as PCA or t-SNE are often applied before using DBSCAN on high-dimensional data.

Conclusion

Through experiments on both synthetic and real-world datasets, DBSCAN was shown to effectively identify clusters of arbitrary shape without requiring the number of clusters to be specified in advance. Its ability to label low-density points as noise further distinguishes it from centroid-based algorithms such as K-Means, which assume spherical clusters and assign all data points to a group.

However, DBSCAN's performance was found to be highly dependent on the choice of its parameters, ϵ and `min_samples`, and it struggled when clusters exhibited varying densities. In contrast, HDBSCAN addressed this limitation by adaptively determining density thresholds, yielding more stable results and better separation of clusters with differing densities.

DBSCAN is most commonly applied in scenarios where data exhibit irregular shapes, variable densities, or contain noise that should not be forced into clusters. It is widely used in spatial data analysis, such as identifying geographic regions of interest or detecting hotspots in GPS data, as well as in anomaly and outlier detection in network security, manufacturing, and sensor data. DBSCAN is also applied in image processing, genomics, and astronomical data analysis, where natural groupings are complex and not easily captured by algorithms assuming spherical clusters or uniform densities.

Lab 3 - Hierarchial Clustering

References

1. University of Orleans. (2025). Machine Learning Lab Manual Series (TPs): K-Means, DBSCAN, Hierarchical Clustering, Regression, KNN, Logistic Regression, Gradient Descent, and Perceptron. Department of Computer Science. Prepared by Prof. Prof. Frédéric Ros
2. McKinney, W. (2022). Python for Data Analysis. O'Reilly Media.